Dynamic Spectral Decoupling for Time-Series Forecasting

Jaehyung Lee
Pohang University of Science and Technology
Pohang, Gyeongbuk, Republic of Korea
anthi7xs@gmail.com

Seungha Hong
Pohang University of Science and Technology
Pohang, Gyeongbuk, Republic of Korea
shhong97@postech.ac.kr

Seongje Lee
Pohang University of Science and Technology
Pohang, Gyeongbuk, Republic of Korea
lsj720@postech.ac.kr

Suyeon Kim
Kyungpook National University
Bukgu, Daegu, Republic of Korea
kimsu55@knu.ac.kr

SeongKu Kang*
Korea University
Seongbuk-gu, Seoul, Republic of Korea
seongkukang@korea.ac.kr

Hwanjo Yu*
Pohang University of Science and Technology
Pohang, Gyeongbuk, Republic of Korea
hwanjoyu@postech.ac.kr

Abstract

While frequency-domain approaches have advanced time-series forecasting, many existing methods process spectral components through a unified pathway, despite substantial energy disparities across frequencies. High-energy components can dominate the learning process, making it difficult to capture weaker residual dynamics that may contain both predictable structure and stochastic fluctuations. To address this issue, we propose Dynamic Spectral Decoupling (DSD), a model-agnostic module that separates frequency components based on instance-wise energy concentration. DSD uses a Lorenz-curve-based adaptive mechanism to determine instance-wise separation boundaries, separates dominant components from the residual spectrum, and assigns the resulting components to separate prediction pathways. This design reduces interference between energy scales without assuming that all low-energy components are equally useful. Extensive experiments on five real-world datasets show that DSD improves the average performance of diverse forecasting backbones and outperforms existing model-agnostic methods. Our code is available at https://github.com/anthi7-google/DSD.

CCS Concepts

• **Mathematics of computing** → Time-series analysis.

Keywords

Time-series forecasting, Spectral analysis, Frequency decomposition, Dynamic spectral decoupling, Model-agnostic learning

ACM Reference Format:
Jaehyung Lee, Seungha Hong, Seongje Lee, Suyeon Kim, SeongKu Kang, and Hwanjo Yu. 2026. Dynamic Spectral Decoupling for Time-Series Forecasting. In *Proceedings of the 35th ACM International Conference on Information and Knowledge Management (CIKM '26), November 07–11, 2026, Rome, Italy.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3799682.3840762

*Corresponding authors.

CIKM '26, Rome, Italy

ACM ISBN 979-8-4007-2539-5/2026/11
https://doi.org/10.1145/3799682.3840762

1 Introduction

Time-series forecasting is a fundamental task in time-series analysis, with applications in smart manufacturing [7], healthcare [11], and traffic prediction [6]. Recent forecasting models have advanced rapidly through Transformer-based architectures and channel-wise modeling strategies [19, 22, 37]. Alongside these developments, frequency-domain approaches have been increasingly explored to capture periodicity, spectral structure, and long-range temporal patterns in time-series.

When real-world time series are transformed into the frequency domain by a Fourier transform, their spectral energy is often unevenly distributed across frequencies. A small number of dominant components may account for a large portion of the total energy, while the remaining components form weaker residual dynamics. These residual components may contain both predictable structure and stochastic fluctuations, but a unified modeling pathway can bias learning toward dominant components and make weaker residual dynamics harder to capture. For example, in household electricity consumption, large appliances such as air conditioners or refrigerators can produce dominant energy patterns, whereas smaller devices such as lighting may induce weaker but still regular variations [27]. Although such variations have smaller amplitudes, they can still contribute to forecasting accuracy when they contain temporal regularity. Figure 1 illustrates this issue with an example. In the time domain, the full signal is largely shaped by high-energy components, making weaker residual variations less visible. The model prediction partially follows the dominant waveform but misses part of the weaker residual dynamics.

Various strategies have been explored to leverage frequency information, ranging from hand-crafted filtering methods [13, 26] to models that incorporate spectral representations within neural architectures [31, 34]. Recent studies have further noted that models do not learn all frequency components equally well, especially when dominant spectral components obscure weaker components during training [23]. Amplifier also addresses the under-representation of low-energy components in time-series forecasting, emphasizing

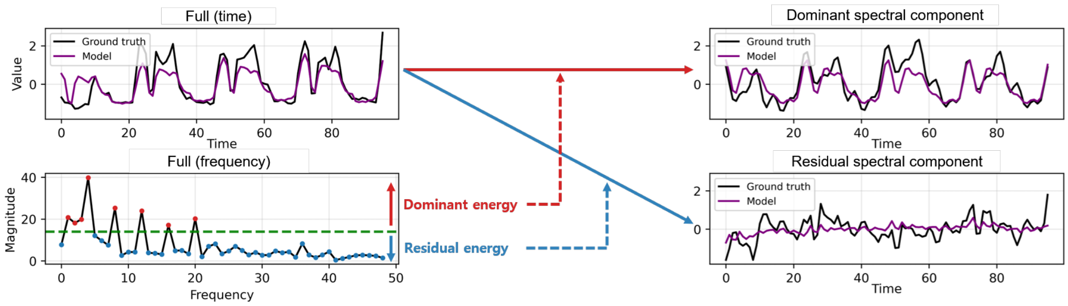

Figure 1: Example from the Electricity dataset on a single channel. Ground truth and DLinear prediction are decomposed into dominant and residual spectral components based on frequency magnitude. The prediction mainly follows the dominant waveform but misses part of the weaker residual dynamics. Here, dominant and residual components refer to spectral energy levels, not frequency ranges.

the importance of energy-scale differences in spectral modeling [8]. DSD addresses a related issue from a different perspective: rather than focusing only on correcting weaker components, it separates dominant and residual spectral components according to instance-wise energy concentration and routes them through different prediction pathways. This design reduces interference between energy scales without relying on uniform amplification of low-energy components.

Building on this perspective, we propose Dynamic Spectral Decoupling (DSD), which adaptively separates spectral components according to their energy concentration rather than processing the entire spectrum through a single uniform pathway. A key requirement of this separation is to adapt to how concentrated the spectral energy is within each input instance, rather than selecting a fixed number or ratio of frequencies for all samples. For this purpose, DSD uses a Lorenz-curve-based concentration score derived from the cumulative spectral magnitude distribution to determine an instance-wise separation boundary. The separated dominant components are routed through auxiliary prediction pathways, while the remaining residual spectrum is assigned to the backbone. We instantiate these auxiliary pathways with lightweight MLP predictors to keep DSD compact as a plug-in module. This design allows dominant and residual components to be modeled through different prediction pathways, reducing interference between energy scales. Importantly, DSD does not assume that all residual components are informative. The residual spectrum may contain both weak structured dynamics and stochastic fluctuations. By separating dominant components first, DSD allows the backbone to focus on the remaining residual dynamics without forcing the same pathway to jointly fit components with substantially different energy scales. We further analyze the spectral entropy of the separated components in Appendix A, which provides additional evidence that dominant and residual components often exhibit different structural characteristics.

The contributions of this paper are as follows:

- We propose Dynamic Spectral Decoupling (DSD), a frequency-domain module that adaptively separates dominant and residual spectral components according to instance-wise energy concentration, with optional multi-stage extension for finer residual decomposition.
- We design DSD as a model-agnostic plug-in module that assigns dominant and residual spectral components to separate prediction pathways, reducing interference between energy scales without uniformly amplifying residual components. We instantiate the auxiliary pathways with lightweight MLP predictors to keep the added branches compact.
- We provide empirical analyses and controlled comparisons showing that the proposed adaptive decoupling is more effective than fixed spectral separation rules and that the gains are not simply due to increased model capacity.
- We validate DSD on five real-world forecasting datasets with diverse backbone models, showing improved average performance across architectures and competitive performance against representative model-agnostic methods.

2 Related Work

Time-series forecasting has evolved from classical statistical methods [3] to recurrent neural network models [9, 16, 24, 35]. More recently, Transformer architectures [28] have become widely used for capturing long-range temporal dependencies, with LogTrans [15] and Informer [37] demonstrating the scalability of attention mechanisms for long-sequence forecasting. DLinear [36] and PatchTST [22] popularized channel-independent modeling, achieving strong performance with remarkable efficiency and simplicity. Building on this trend, subsequent studies [4, 21] introduced further architectural refinements. In parallel, iTransformer [19] revisited channel-dependent designs, while LLM-based approaches [10, 18, 40] transfer advances from natural language processing to time-series forecasting.

While most of these methods operate mainly in the time domain, another line of research has focused on frequency-domain representations for forecasting. Frequency-based approaches for time-series forecasting can be broadly categorized into two types: those that use frequency information as a preprocessing step and those that integrate it as part of the model. Preprocessing-based methods aim to suppress noise by removing certain frequency components from the input sequence, often relying on domain knowledge to predefine the filter bands. For example, Kumar et al. [13] applied an FFT-based bandpass filter to emphasize QRS complexes in ECG signals, and Song et al. [26] used Fourier transform denoising with thresholding to suppress market noise in financial time-series. However, such methods are difficult to generalize when the spectral distribution differs across datasets.

In contrast, recent approaches have incorporated frequency information directly into the model's internal mechanism. Several notable frequency-based models include FNet [14], which replaces self-attention with a Fourier transform, FEDformer [38], which uses frequency-enhanced attention to capture long-range patterns, TimesNet [30], which leverages high-amplitude frequency components through CNN-based extraction, FiLM [39], which integrates frequency-domain projections to enhance memory modeling and suppress noise, FreTS [34], which transforms the input into the frequency domain and processes the spectrum with an MLP, and FITS [31], which performs interpolation over the full spectrum to generate future signals. Fredformer [23] converts the input into the frequency domain and applies patch-based tokenization along the spectral axis with a Transformer backbone. FilterNet [33] transforms the input via FFT and applies learnable frequency filters to selectively amplify or attenuate spectral components. FreDF [29] transforms the label and prediction sequences into the frequency domain and introduces an auxiliary frequency-domain loss to correct the bias inherent in the direct forecasting paradigm.

Recent studies have also begun to examine imbalance in frequency-domain learning. Fredformer [23] addresses frequency-domain bias through spectral tokenization and normalization, while Amplifier [8] brings attention to neglected low-energy components through energy amplification and restoration. These studies highlight that spectral components are not learned uniformly and that energy-scale differences are important for forecasting. In contrast, DSD uses instance-wise energy concentration to separate dominant and residual components and route them through different prediction pathways, providing a complementary way to address energy-scale imbalance. This model-agnostic design reduces energy-scale interference while remaining compatible with diverse forecasting backbones.

3 Preliminaries

3.1 Problem Formulation

We formulate the time-series forecasting task as learning a mapping from an input window $\mathbf{X}_{t-L+1:t} \in \mathbb{R}^{L\times C}$ to a target window $\mathbf{X}_{t+1:t+H} \in \mathbb{R}^{H\times C}$, where L is the input length, H is the prediction horizon, and C is the number of variables. For simplicity, we denote each instance by $\mathbf{x} = \mathbf{X}_{t-L+1:t} \in \mathbb{R}^{L\times C}$, and its corresponding prediction by $\mathbf{y} = \mathbf{X}_{t+1:t+H} \in \mathbb{R}^{H\times C}$.

3.2 Spectral Energy Concentration

For adaptive spectral separation, the model needs an instance-wise criterion that measures how concentrated spectral energy is across frequency components. A spectrum dominated by a few high-magnitude frequencies should be separated differently from one whose energy is spread more evenly. To capture this difference, we use the Lorenz curve [25], which describes the cumulative distribution of ordered non-negative values.

Given ordered non-negative values $a_{(1)} \leq \cdots \leq a_{(n)}$, we define the partial sums as

$$S_0 = 0, \qquad S_k = \sum_{j=1}^{k} a_{(j)}, \quad k = 1, \ldots, n. \tag{1}$$

The Lorenz curve is given by the points $\left(\frac{k}{n}, \frac{S_k}{S_n}\right)$ for $k = 0, \ldots, n$. We summarize the area under this curve using a right-rectangle approximation:

$$B_{\mathrm{rr}} = \frac{1}{n}\sum_{k=1}^{n} \frac{S_k}{S_n}. \tag{2}$$

The resulting concentration score is defined as

$$\gamma = 1 - 2B_{\mathrm{rr}} = 1 - \frac{2}{n}\sum_{k=1}^{n} \frac{S_k}{S_n}. \tag{3}$$

A larger γ indicates that the magnitude distribution is more concentrated around a few dominant components, while a smaller value indicates a more even distribution. In DSD, this score is used as an adaptive boundary indicator and is further calibrated before mask construction.

4 Method

DSD (Dynamic Spectral Decoupling) is a model-agnostic module that separates dominant spectral components from the residual spectrum and can extend the Lorenz-curve-based separation over d stages for progressive residual refinement. Given an input sequence $\mathbf{x}$, applying $\mathcal{D}$ over d stages yields d decoupled components and the remaining residual:

$$\{\mathbf{x}_{\mathrm{dec}}^{(1)}, \mathbf{x}_{\mathrm{dec}}^{(2)}, \ldots, \mathbf{x}_{\mathrm{dec}}^{(d)}, \mathbf{x}_{\mathrm{res}}\} = \mathcal{D}(\mathbf{x}). \tag{4}$$

Each decoupled component $\mathbf{x}_{\mathrm{dec}}^{(j)}$ is passed to an auxiliary predictor $g^{(j)}$, while the residual $\mathbf{x}_{\mathrm{res}}$ is fed into the backbone model f:

$$\hat{\mathbf{y}}_{\mathrm{dec}}^{(j)} = g^{(j)}(\mathbf{x}_{\mathrm{dec}}^{(j)}), \qquad \hat{\mathbf{y}}_{\mathrm{res}} = f(\mathbf{x}_{\mathrm{res}}). \tag{5}$$

The final prediction is reconstructed as

$$\hat{\mathbf{y}} = \hat{\mathbf{y}}_{\mathrm{res}} + \sum_{j=1}^{d} \hat{\mathbf{y}}_{\mathrm{dec}}^{(j)}. \tag{6}$$

In this design, the auxiliary predictors $g^{(j)}$ model the decoupled dominant components, while the backbone f models the remaining residual spectrum. By routing components with different energy scales through different pathways, DSD reduces interference between dominant components and residual dynamics without assuming that all residual components are uniformly informative. This separation also reduces the risk that high-amplitude components dominate MSE-based optimization, making weaker residual dynamics less influential during parameter updates. Since the auxiliary predictors are instantiated as compact MLPs, the added prediction branches introduce limited computational overhead. Figure 2 illustrates the overall architecture of DSD.

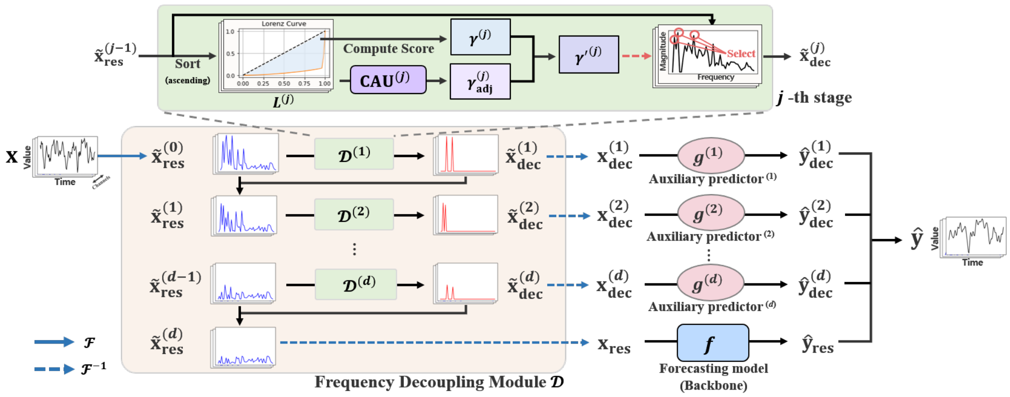

Figure 2: Overview of the proposed DSD module extended to d spectral decoupling stages. The blue arrows indicate the Fourier transform $\mathcal{F}$ and its inverse $\mathcal{F}^{-1}$, which convert between the time and frequency domains.

4.1 Frequency Decoupling Module

The frequency decoupling module takes the current residual spectrum as input and separates dominant spectral components using an instance-wise energy concentration criterion. Since each separation step produces both a decoupled component and an updated residual spectrum, the same procedure can be extended over d stages to progressively refine the residual spectrum.

A straightforward way to construct the separation is to use fixed rules. We consider three representative alternatives: (i) partitioning the spectrum into predefined frequency bands, (ii) applying global magnitude thresholds estimated from the training set, and (iii) selecting a fixed number of top-magnitude frequency components.

However, these rules do not adapt to the spectral distribution of each input instance. Predefined frequency bands depend on spectral position rather than energy concentration. Global magnitude thresholds provide dataset-level consistency, but cannot adjust to sample-wise variations in magnitude scale. Fixed top-k selection accounts for magnitude ordering, but it still selects the same number of components regardless of whether the spectrum is highly concentrated or nearly flat.

In contrast, DSD can be viewed as an adaptive top-k-like separation, where the number of selected components varies according to the spectral concentration of each instance. We use the Lorenz-curve-based concentration score to obtain this instance-wise criterion and convert it into a channel-wise threshold over cumulative top-k magnitudes. The resulting soft mask routes spectral components to the decoupled branch without relying on a manually fixed k.

The internal procedure of the decoupling module $\mathcal{D}$ is as follows. The decoupling module first computes the initial spectrum $\tilde{\mathbf{x}}_{\text{res}}^{(0)} = \mathcal{F}(\mathbf{x})$ by applying a Fourier transform to the input sequence $\mathbf{x}$. It then applies the same separation procedure over d stages to refine the residual spectrum. Specifically, at each stage $\mathcal{D}^{(j)}$, the decoupling module takes the residual spectrum $\tilde{\mathbf{x}}_{\text{res}}^{(j-1)} \in \mathbb{C}^{F\times C}$ as input and constructs a soft mask $W^{(j)}$ that determines how much of each spectral component is routed to the decoupled branch. Here, F denotes the number of frequency bins and C is the number of channels.

$$\tilde{\mathbf{x}}_{\text{dec}}^{(j)} = \mathcal{D}^{(j)}(\tilde{\mathbf{x}}_{\text{res}}^{(j-1)}) = W^{(j)} \odot \tilde{\mathbf{x}}_{\text{res}}^{(j-1)}, \tag{7}$$

$$\tilde{\mathbf{x}}_{\text{res}}^{(j)} = (1 - W^{(j)}) \odot \tilde{\mathbf{x}}_{\text{res}}^{(j-1)}, \quad j = 1, \ldots, d. \tag{8}$$

Finally, the decoupled and residual spectra are transformed back to the time domain as:

$$\mathbf{x}_{\text{dec}}^{(j)} = \mathcal{F}^{-1}(\tilde{\mathbf{x}}_{\text{dec}}^{(j)}), \qquad \mathbf{x}_{\text{res}} = \mathcal{F}^{-1}(\tilde{\mathbf{x}}_{\text{res}}^{(d)}). \tag{9}$$

In the following, we present the construction of the soft mask $W^{(j)}$ in detail.

4.1.1 DC Handling. As the first step, we remove the DC (direct current) component, which represents the global mean and does not contain temporal variations. The DC bin ($k = 0$) is excluded from energy analysis. After removing it, we obtain the non-DC spectrum:

$$\tilde{\mathbf{x}}_{\text{res}\setminus 0}^{(j-1)} = \tilde{\mathbf{x}}_{\text{res}}^{(j-1)}[1:] \in \mathbb{C}^{N_f \times C}, \tag{10}$$

where $N_f = F - 1$ is the number of non-DC bins. All subsequent processes are performed only on this non-DC part. During mask construction, a DC weight of 1 is prepended so that the DC component is routed to the decoupled branch and excluded from the residual spectrum after the first stage.

4.1.2 Lorenz-curve-based Energy Analysis. To construct an instance-wise separation boundary, we measure the concentration of spectral magnitudes across the non-DC frequencies. We first compute the magnitude spectrum:

$$\mathbf{m}^{(j)} = \left|\tilde{\mathbf{x}}_{\text{res}\setminus 0}^{(j-1)}\right| \in \mathbb{R}^{N_f \times C}, \tag{11}$$

and sort it in ascending order along the frequency axis for each channel, $\mathbf{m}_{(1)}^{(j)} \le \cdots \le \mathbf{m}_{(N_f)}^{(j)}$. Let the cumulative sums of the sorted magnitudes be

$$S_0^{(j)} = 0, \qquad S_k^{(j)} = \sum_{i=1}^{k} \mathbf{m}_{(i)}^{(j)}, \quad k = 1, \ldots, N_f. \tag{12}$$

We then define the normalized cumulative magnitude, corresponding to the Lorenz curve, as:

$$L_k^{(j)} = \frac{S_k^{(j)}}{S_{N_f}^{(j)}} \in [0,1]^C, \quad k = 1, \dots, N_f. \tag{13}$$

We then compute the Lorenz-curve-based concentration score as

$$\gamma^{(j)} = 1 - \frac{2}{N_f} \sum_{k=1}^{N_f} \frac{S_k^{(j)}}{S_{N_f}^{(j)}} = 1 - \frac{2}{N_f} \sum_{k=1}^{N_f} L_k^{(j)}. \tag{14}$$

4.1.3 Concentration Adjustment Unit. The raw concentration score provides an instance-wise measure of spectral concentration, but a fixed deterministic mapping from this score to a separation boundary may be too rigid for different datasets and channels. To calibrate this boundary in a data-dependent but lightweight manner, we introduce a Concentration Adjustment Unit (CAU). Given the normalized cumulative magnitude values $\{L_k^{(j)}\}_{k=1}^{N_f}$ from the residual spectrum, CAU produces a channel-wise adjustment term:

$$\gamma_{\text{adj}}^{(j)} = \text{CAU}^{(j)}\left(\{L_k^{(j)}\}_{k=1}^{N_f}\right) \in \mathbb{R}^C. \tag{15}$$

Each CAU is implemented as a two-layer MLP with a final Tanh activation. The Tanh activation bounds the adjustment and allows both positive and negative corrections around zero. The adjusted concentration score is then computed as

$$\gamma'^{(j)} = \gamma^{(j)} + \gamma_{\text{adj}}^{(j)}. \tag{16}$$

We clip $\gamma'^{(j)}$ into $[0, 1]$ to ensure that it remains a valid boundary ratio. During backpropagation, we apply the straight-through estimator (STE) [2] to the clipping operation so that gradients can pass through the boundary.

4.1.4 Selection Mask Construction. For each separation stage, we construct a soft selection mask that routes dominant spectral components to the decoupled branch while leaving the remaining components in the residual branch. We first compute the cumulative magnitude explained by the top-k frequency bins by summing $\mathbf{m}^{(j)} \in \mathbb{R}^{N_f \times C}$ in descending order:

$$E_k^{(j)} = \sum_{i=1}^{k} \mathbf{m}_{(N_f - i + 1)}^{(j)}, \quad k = 1, \dots, N_f. \tag{17}$$

The total magnitude is given by $E_{\text{total}}^{(j)} := E_{N_f}^{(j)} \in \mathbb{R}^C$. Using the adjusted concentration score $\gamma'^{(j)}$, we define an ideal binary mask that selects the top frequency bins whose cumulative magnitude does not exceed the channel-wise threshold:

$$b_k^{(j)} = \mathbb{I}\left(\gamma'^{(j)} \odot E_{\text{total}}^{(j)} > E_k^{(j)}\right), \tag{18}$$

where $\mathbb{I}(\cdot)$ denotes the indicator function. Since hard selection is non-differentiable, we approximate this ideal binary mask with a differentiable soft selection mask:

$$b_k^{(j)} \approx w_k^{(j)} = \sigma\left(\gamma'^{(j)} \odot E_{\text{total}}^{(j)} - E_k^{(j)}\right), \tag{19}$$

where $w_k^{(j)} \in \mathbb{R}^C$ and $w^{(j)} = [w_k^{(j)}]_{k=1}^{N_f} \in \mathbb{R}^{N_f \times C}$.

Finally, we prepend the DC bin to construct the full mask $W^{(j)}$. Since the DC bin ($k = 0$) represents the global mean component and is not involved in the Lorenz-curve-based selection, we set its weight to 1 and concatenate it with the non-DC mask:

$$W^{(j)} = \text{concat}\left(1,\ \mathbf{w}^{(j)}\right) \in \mathbb{R}^{F \times C}. \tag{20}$$

This soft mask is used to route dominant spectral components to the decoupled branch at each stage, as in Eq. 7.

4.1.5 Computational Complexity. The frequency decoupling module consists of FFT, sorting, cumulative-magnitude computation, and mask construction. Since FFT and sorting dominate the cost, its overall complexity is $O(CL \log L)$, the same asymptotic order as FFT-based spectral processing. A detailed derivation is provided in Appendix B.

Moreover, transformer-based models commonly used in time-series forecasting have a self-attention complexity of $O(L^2 d_{\text{model}})$, where d_{model} denotes the hidden dimension of the transformer block. In channel-independent designs, the same self-attention operation is repeated for each channel, resulting in $O(CL^2 d_{\text{model}})$. Our module adds an additional $O(CL \log L)$ cost under both settings. In the channel-dependent configuration, the backbone self-attention operates at $O(L^2 d_{\text{model}})$, so the overall complexity becomes $O(L^2 d_{\text{model}} + CL \log L)$, an asymptotic form consistent with transformer models that employ FFT-based preprocessing. In the channel-independent configuration, the backbone is dominated by $O(CL^2 d_{\text{model}})$, and the added $O(CL \log L)$ term does not change the asymptotic order of the backbone.

4.2 Loss Function

The model is trained by supervising its predictions against the target sequence decomposed by the same decoupling module $\mathcal{D}$. We apply $\mathcal{D}$ to the ground truth target $\mathbf{y}$ once to obtain:

$$\{\mathbf{y}_{\text{dec}}^{(1)}, \mathbf{y}_{\text{dec}}^{(2)}, \dots, \mathbf{y}_{\text{dec}}^{(d)}, \mathbf{y}_{\text{res}}\} = \mathcal{D}(\mathbf{y}). \tag{21}$$

Letting $\hat{\mathbf{y}} = \hat{\mathbf{y}}_{\text{res}} + \sum_{j=1}^{d} \hat{\mathbf{y}}_{\text{dec}}^{(j)}$, the final loss is defined as the sum of the component and residual supervision terms:

$$\mathcal{L} = \sum_{j=1}^{d} \text{MSE}\left(\hat{\mathbf{y}}_{\text{dec}}^{(j)},\ \mathbf{y}_{\text{dec}}^{(j)}\right) + \text{MSE}\left(\hat{\mathbf{y}}_{\text{res}},\ \mathbf{y}_{\text{res}}\right). \tag{22}$$

5 Experiments

5.1 Experimental Settings

In this section, we describe the experimental settings, including the datasets and forecasting backbones used for evaluation. Implementation details are provided in Appendix C.

5.1.1 Datasets. We evaluate our model on five commonly used multivariate time-series benchmarks: ETT, Weather, ExchangeRate, Electricity, and Traffic. These datasets cover multiple application domains, including energy systems, meteorology, financial exchange rates, electricity consumption, and transportation. A summary of the datasets is provided in Appendix D. All datasets are split into training, validation, and test sets in a 7:2:1 ratio. We evaluate forecasting performance across prediction horizons $H \in \{96, 168, 336, 720\}$ with a fixed input sequence length of $L = 96$ across all backbones.

Table 1: Average MSE over four prediction horizons for each dataset and backbone. Each entry reports baseline → DSD, with full horizon-wise MSE and MAE results provided in Appendix G.

Dataset	SCINet	DLinear	PatchTST	FITS	iTransformer	Fredformer	FilterNet	Amplifier
ETTh1	0.536→**0.454**	**0.443**→0.450	0.446→**0.443**	0.460→**0.450**	0.455→**0.440**	0.459→**0.444**	0.474→**0.446**	0.465→**0.445**
ETTh2	0.177→**0.135**	0.138→**0.134**	0.141→**0.134**	0.145→**0.135**	0.155→**0.133**	0.141→**0.134**	0.153→**0.135**	0.147→**0.134**
ETTm1	0.442→**0.401**	**0.389**→0.403	**0.390**→0.398	0.405→**0.402**	0.411→**0.397**	0.409→**0.397**	0.422→**0.399**	0.419→**0.401**
ETTm2	0.108→**0.106**	0.108→**0.106**	0.108→**0.106**	0.112→**0.106**	0.111→**0.106**	0.111→**0.106**	0.114→**0.106**	0.112→**0.106**
Weather	0.282→**0.253**	0.278→**0.250**	0.255→**0.255**	0.277→**0.251**	0.263→**0.255**	0.270→**0.256**	**0.256**→0.257	**0.253**→0.256
Exchange	0.210→**0.153**	0.170→**0.142**	0.298→**0.152**	0.180→**0.142**	0.180→**0.153**	0.176→**0.152**	0.204→**0.153**	0.181→**0.153**
Electricity	0.216→**0.191**	0.202→**0.192**	0.201→**0.192**	0.208→**0.192**	0.203→**0.193**	0.208→**0.194**	0.201→**0.193**	0.195→**0.193**
Traffic	0.469→**0.430**	0.505→**0.430**	0.463→**0.438**	0.523→**0.430**	0.440→**0.438**	0.434→**0.431**	0.473→**0.444**	0.463→**0.445**
Avg.	0.305→**0.265**	0.279→**0.263**	0.288→**0.265**	0.289→**0.263**	0.277→**0.264**	0.276→**0.264**	0.287→**0.266**	0.279→**0.267**

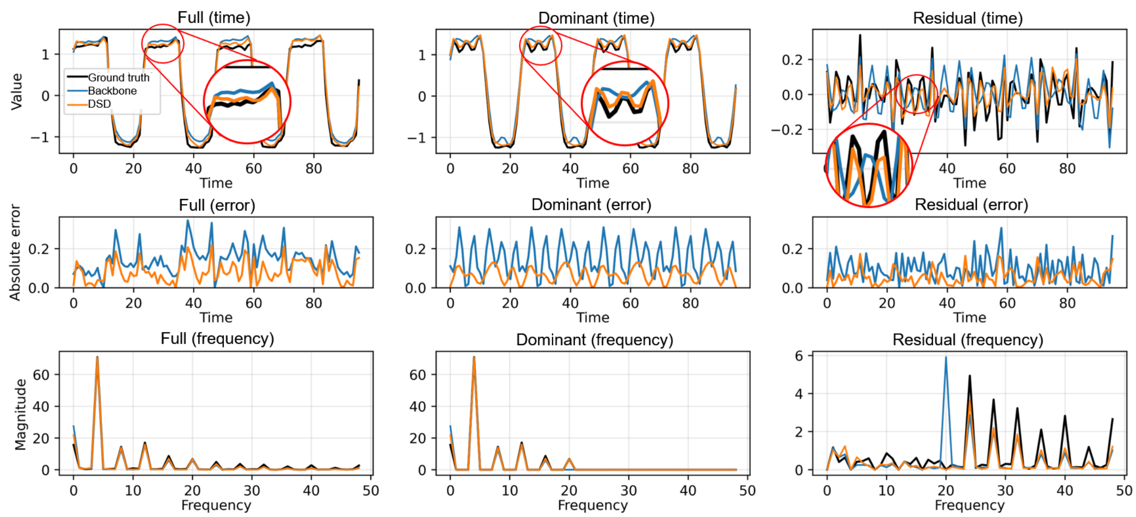

Figure 3: Example predictions on a single channel from the Electricity dataset, decomposed into dominant and residual spectral components based on spectral energy. We use PatchTST as the backbone and compare the original backbone prediction with the prediction of the same backbone equipped with DSD.

5.1.2 Forecasting Backbones. We use the following representative forecasting models as backbones for evaluating DSD.

- **SCINet** [17] recursively downsamples time-series into subsequences and applies convolution and interaction to capture temporal features at multiple resolutions.
- **DLinear** [36] decomposes time-series into trend and residual components and applies channel-independent linear projections for efficient forecasting.
- **PatchTST** [22] segments time-series into patches and applies channel-independent self-attention to model long-term dependencies efficiently.
- **FITS** [31] transforms time-series into the frequency domain and performs interpolation over the entire spectrum to generate future signals.
- **iTransformer** [19] employs an inverted Transformer design to better capture multivariate dependencies.
- **Fredformer** [23] transforms inputs into the frequency domain and applies patch-based tokenization along the spectral axis.
- **FilterNet** [33] transforms inputs via FFT and uses learnable spectral filters to modulate specific frequency components.
- **Amplifier** [8] improves forecasting by amplifying underrepresented low-energy components in the frequency domain.

5.2 Main Results

5.2.1 Effect of DSD on Backbone Models. We evaluate the effectiveness of DSD as a plug-in method by applying it to eight representative backbone models introduced above. Table 1 summarizes the dataset-wise average MSE over four prediction horizons for each backbone, while the full horizon-wise MSE and MAE results are provided in Appendix G. In terms of average MSE, DSD improves all evaluated backbones. On average, SCINet achieves the largest relative gain (≈13.4%), followed by FITS (≈7.47%), PatchTST

Table 2: Forecasting performance (MSE) of DSD compared with other model-agnostic methods (FAN, SAN, RevIN). DSD achieves the best average performance across all evaluated backbones, demonstrating its robustness and general applicability.

Models	DLinear					PatchTST					FITS					iTransformer				
Methods	DSD	FAN	SAN	RevIN	No	DSD	FAN	SAN	RevIN	No	DSD	FAN	SAN	RevIN	No	DSD	FAN	SAN	RevIN	No
ETTh1	0.450	0.452	0.455	0.462	**0.443**	**0.443**	0.447	0.450	0.461	0.446	**0.450**	0.454	0.457	0.460	0.460	**0.440**	0.442	**0.440**	0.455	0.455
Weather	0.250	0.256	**0.249**	0.271	0.278	0.255	0.260	**0.250**	0.259	0.255	0.251	0.256	**0.249**	0.277	0.277	0.255	0.259	**0.246**	0.259	0.263
Exchange	**0.142**	0.150	0.166	0.191	0.170	**0.152**	0.153	0.180	0.180	0.298	**0.142**	0.149	0.166	0.180	0.180	**0.153**	**0.153**	0.168	0.184	0.180
Electricity	**0.192**	0.195	0.200	0.207	0.202	**0.192**	0.195	0.200	0.196	0.201	**0.192**	0.198	0.208	0.208	0.208	**0.193**	0.197	0.201	0.199	0.203
Traffic	**0.430**	0.431	0.460	0.519	0.505	**0.438**	0.439	0.459	0.469	0.463	**0.430**	0.431	0.475	0.523	0.523	0.438	0.440	0.442	**0.437**	0.440
Avg	**0.293**	0.297	0.306	0.330	0.320	**0.296**	0.299	0.308	0.313	0.333	**0.293**	0.298	0.311	0.330	0.330	**0.296**	0.298	0.299	0.307	0.308

(≈6.73%), FilterNet (≈6.44%), and iTransformer (≈5.04%), while DLinear (≈4.72%), Amplifier (≈4.30%), and Fredformer (≈4.04%) also show modest but consistent gains from DSD. Notably, FITS, Fredformer, FilterNet, and Amplifier, all of which incorporate frequency-domain or spectral-energy modeling, still benefit from DSD, suggesting that DSD provides a complementary intervention to existing spectral modeling strategies. Across datasets, the largest gains are observed on ExchangeRate (≈16.0%), while DSD also improves the average performance on the other datasets. These results demonstrate that DSD can enhance diverse backbones without modifying their internal architectures, indicating its model-agnostic nature. Additional multi-seed results in Appendix F show that these gains are not merely due to a favorable random initialization.

5.2.2 Comparison with Other Model-agnostic Methods. We further compare DSD with three representative model-agnostic add-on methods for improving time-series forecasting backbones. RevIN [12] normalizes each instance and restores the prediction to the original scale through an inverse mapping. SAN [20] predicts slice-specific statistics to improve adaptability to distributional changes. FAN [32] removes fixed frequency components identified in the Fourier domain. Table 2 compares DSD with other model-agnostic methods across four representative backbones. Although some existing methods perform competitively on individual datasets, DSD achieves the best average MSE for all evaluated backbones. This suggests that instance-wise spectral decoupling provides a more reliable plug-in intervention than normalization-based adjustment or fixed frequency filtering.

5.3 Analysis and Ablation Studies

We organize the following analyses around the main design questions of DSD. The visualization first examines how DSD affects predicted spectral components. We then test whether adaptive boundary selection can be replaced by fixed top-k selection, fixed filter-bank partitioning, or global-threshold rules. Next, we isolate Lorenz-guided decoupling from CAU calibration and analyze the effect of the number of decoupling stages. Finally, we examine whether the gains are mainly explained by auxiliary predictor capacity.

5.3.1 Visualization Analysis. We provide a qualitative visualization to examine how DSD affects the spectral components predicted by the backbone. Figure 3 compares the ground truth, backbone prediction, and DSD-equipped prediction after decomposing them into dominant and residual spectral components based on spectral energy. In the residual component, the backbone misses a substantial portion of weaker variations, whereas DSD produces predictions that better follow some of these overlooked components. As a result, the DSD-equipped prediction is closer to the ground truth than the original backbone prediction, especially in the residual component and its error pattern. These results suggest that DSD helps the backbone capture a broader range of spectral components, reducing its tendency to focus mainly on a few high-energy components and producing more detailed predictions.

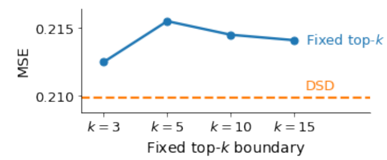

Figure 4: Additional ablation on adaptive boundary selection using PatchTST on the Weather dataset at prediction horizon 168. All tested fixed top-k choices yield higher MSE than the adaptive DSD boundary.

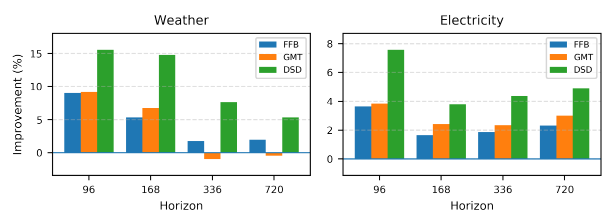

Figure 5: Ablation study on the frequency decoupling strategy. The figure reports relative MSE improvement over the DLinear backbone on Weather and Electricity. DSD achieves the strongest overall improvement compared with FFB and GMT. The corresponding absolute MSE values are provided in Appendix E.

5.3.2 Effect of Adaptive Boundary Selection. We first test whether DSD's adaptive boundary is necessary, or whether fixed top-k selection with a fixed number of frequency components is sufficient. For this analysis, we use PatchTST on the Weather dataset with a prediction horizon of 168. As shown in Figure 4, all tested fixed k values yield higher MSE than DSD. This result indicates that the gain is not obtained by finding a favorable fixed number of dominant frequency components. Instead, DSD benefits from adapting the effective selection size to the spectral concentration of each input.

Table 3: CAU ablation results in MSE. DSD* **denotes DSD without CAU.**

Method	Weather				Electricity			
	96	168	336	720	96	168	336	720
DLinear+DSD	**0.1741**	**0.2089**	**0.2771**	**0.3401**	**0.1804**	**0.1757**	**0.1886**	**0.2217**
DLinear+DSD*	0.1754	0.2125	0.2793	0.3432	0.1809	0.1770	0.1895	0.2226
DLinear	0.2062	0.2451	0.3000	0.3592	0.1952	0.1826	0.1972	0.2331

5.3.3 Comparative Analysis of Frequency Decoupling Methods. We next examine whether the benefit of DSD comes merely from splitting the spectrum, or from the proposed instance-wise energy-aware separation rule. Fixed Filter Bank (FFB), built on the classical filter-bank principle of subband decomposition [1], partitions the spectrum into three predefined low-, middle-, and high-frequency bands, thereby separating components according to spectral position. Global Magnitude Thresholding (GMT), inspired by magnitude-thresholding methods in transform-domain signal denoising [5], instead uses thresholds estimated once from the training set, with normalized boundaries set to one-third and two-thirds of the maximum spectral magnitude observed during training. All other components are kept identical to DSD.

FFB tests whether a simple position-based spectral partition is sufficient, while GMT tests whether a globally fixed magnitude-based rule can replace instance-wise adaptation. However, FFB does not account for spectral energy imbalance, and GMT cannot adjust its separation boundary to each input sequence. As shown in Figure 5, both variants provide only limited improvements over the backbone, whereas DSD achieves the best performance. This suggests that the gains of DSD come not merely from using frequency decoupling, but from combining energy-aware separation with instance-wise adaptive boundary determination.

5.3.4 Effect of Lorenz-guided Decoupling. We then isolate whether the gain mainly comes from the Lorenz-guided decoupling rule itself or from the learnable CAU calibration. To this end, we compare *DSD* with a variant denoted as *DSD**, where CAU is removed while the Lorenz-curve-based decoupling rule is retained. As shown in Table 3, both *DSD* and *DSD** substantially outperform the plain DLinear backbone across all horizons on Weather and Electricity. The strong performance of *DSD** shows that the proposed mechanism does not primarily rely on CAU; rather, the main gain comes from concentration-guided spectral decoupling itself, which separates components according to instance-wise spectral energy concentration.

CAU further improves upon *DSD** in all reported cases, but the gap is relatively moderate compared with the improvement from the backbone to *DSD**. This suggests that CAU mainly acts as a learnable calibration unit for refining the Lorenz-curve-based separation boundary, rather than being the primary source of the gain. Overall, these results support the design choice of using the Lorenz-curve-based concentration score as the core separation criterion and CAU as an auxiliary adjustment mechanism for instance-wise boundary calibration.

5.3.5 Sensitivity to the Number of Decoupling Layers. We analyze the sensitivity of DSD to the number of decoupling layers d. Figure 6 reports the MSE trends on Traffic with the PatchTST backbone and on Weather with the DLinear backbone as d varies, where $d = 0$ denotes the plain backbone without decoupling. Introducing a single decoupling stage already provides a clear improvement over the plain backbone, showing that DSD is effective even in the single-stage setting. Additional stages can further refine the residual spectrum, but the improvement is not monotonic and the best value may vary across datasets and prediction horizons. After the initial gain from $d = 1$, performance remains within a relatively stable range for larger d, indicating that DSD is not highly sensitive to the exact number of stages. These results suggest that a small fixed value of d is sufficient in practice, while multi-stage decoupling serves as an optional extension when additional residual separation is beneficial.

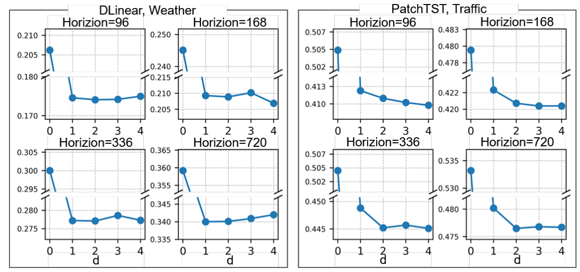

Figure 6: Sensitivity analysis on the number of decoupling layers d in DSD. The performance improves clearly from $d = 0$ to $d = 1$, while larger d values lead to non-monotonic but relatively stable trends across datasets and horizons.

Table 4: Ablation on auxiliary predictor (MSE). DSD^P uses PatchTST as the auxiliary predictor.

Method	Weather				Electricity			
	96	168	336	720	96	168	336	720
PatchTST+DSD	**0.1767**	**0.2099**	0.2856	0.3467	**0.1809**	**0.1757**	0.1889	0.2217
PatchTST+DSD^P	0.1775	0.2118	**0.2775**	0.3509	0.1852	0.1767	**0.1881**	**0.2194**
PatchTST	0.1816	0.2154	0.2789	**0.3447**	0.1955	0.1838	0.1969	0.2284

5.3.6 Ablation on Auxiliary Predictor. We further examine whether the gains of DSD depend on using a stronger auxiliary predictor. To this end, we replace the default lightweight MLP predictor with PatchTST as the auxiliary predictor while keeping all other components and training configurations unchanged. Table 4 shows that replacing the lightweight MLP with PatchTST in the auxiliary branch improves a few cases, but does not yield consistent gains. The compact MLP remains competitive across both Weather and Electricity, suggesting that the observed gains are not simply explained by increasing the capacity of the auxiliary predictor. This supports the interpretation that the main benefit of DSD comes from the decoupling structure rather than merely from adding auxiliary capacity. Given its competitive performance and lower overhead, we adopt the compact MLP auxiliary predictor throughout the main experiments.

6 Discussion

In this work, we showed that explicitly separating spectral energy scales is an effective design choice for time-series forecasting. DSD

Table 5: Spectral entropy of the full spectrum, residual component, and dominant component across datasets. The second-level columns further decompose the residual component into relatively lower- and higher-energy portions.

Dataset	Full	Residual	Dominant	Residual(2nd-level)	Dominant(2nd-level)
ETTh1	3.263	5.291	2.556	5.301	4.751
Weather	3.629	5.046	2.617	5.064	4.189
ExchangeRate	2.633	4.933	2.201	5.193	4.177
Electricity	3.011	5.412	2.655	5.510	5.105
Traffic	3.475	5.309	2.826	5.447	4.779

implements this idea through instance-wise energy-aware spectral decoupling and improves the average performance of diverse backbones, including frequency-based models. Together with the ablation studies, these results indicate that adaptive boundary determination and frequency-domain decoupling are key components, while the gains are not simply explained by auxiliary predictor capacity.

A practical consideration is that DSD introduces additional auxiliary branches, although we instantiate them with compact MLP predictors to limit overhead. This cost may still matter in highly resource-constrained environments such as mobile or edge devices. We focus on supervised offline forecasting; extending DSD to pretrained foundation models and online learning scenarios involves different optimization and adaptation constraints, which we leave for future work.

Acknowledgments

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2024-00335873). This work was also supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government(MSIT) (No.RS-2019-II191906, Artificial Intelligence Graduate School Program(POSTECH)) and ICT Creative Consilience Program through the IITP grant funded by the MSIT (IITP-2026-RS-2020-II201819).

Appendix

A Spectral Entropy Analysis of Dominant and Residual Components

This section analyzes the spectral entropy of the dominant and residual components produced by our frequency partitioning scheme. The goal of this analysis is not to claim that all residual components are informative, but to examine whether components with different energy levels exhibit different spectral characteristics. For a time series segment $\mathbf{x} \in \mathbb{R}^{L \times C}$, we estimate the channel-wise PSD using Welch's method and sum it over channels:

$$\mathrm{PSD}_c(f_k) = \mathrm{Welch}(\mathbf{x}_{:,c})[k], \quad S(f_k) = \sum_{c=1}^{C} \mathrm{PSD}_c(f_k). \tag{23}$$

We normalize this spectrum and compute the spectral entropy as

$$p_k = \frac{S(f_k)}{\sum_j S(f_j)}, \qquad H(\mathbf{x}) = -\sum_k p_k \log p_k. \tag{24}$$

For each dataset, we report the average spectral entropy of the dominant and residual components in Table 5. Across all datasets, the dominant components show lower entropy than the residual components. This indicates that the separated components exhibit different spectral concentration patterns, with dominant components being more concentrated in frequency and residual components being more dispersed.

To examine whether this pattern remains after the first split, we further decompose the residual component. The same directional trend remains: the relatively higher-energy subset has lower entropy than the relatively lower-energy subset, although the gap becomes smaller. This suggests that residual energy-scale differences can persist after a single-stage split, supporting progressive decoupling. Importantly, higher entropy should not be interpreted as evidence that all residual components are useful signals; rather, it indicates distinct spectral characteristics that may require a different modeling pathway from the dominant components.

Table 6: Statistics of the datasets.

Datasets	ETTh1	ETTh2	ETTm1	ETTm2
Variables	7	7	7	7
Length	17,420	17,420	69,680	69,680
Resolution	1 Hour	1 Hour	15 Minutes	15 Minutes
Datasets	**Weather**	**ExchangeRate**	**Electricity**	**Traffic**
Variables	21	8	321	862
Length	52,696	7,588	26,304	17,544
Resolution	10 Minutes	1 Day	1 Hour	1 Hour

Table 7: Absolute MSE values for the frequency decoupling strategy ablation.

Method	Weather				Electricity			
	96	168	336	720	96	168	336	720
Backbone	0.2062	0.2451	0.3000	0.3592	0.1952	0.1826	0.1972	0.2331
FFB	0.1875	0.2320	0.2946	0.3521	0.1881	0.1796	0.1935	0.2277
GMT	0.1872	0.2286	0.3028	0.3608	0.1877	0.1782	0.1926	0.2261
DSD	**0.1741**	**0.2089**	**0.2771**	**0.3401**	**0.1804**	**0.1757**	**0.1886**	**0.2217**

Table 8: Forecasting performance (MSE) as mean ± standard deviation over 5 seeds.

Models		DLinear		PatchTST	
Dataset	Horizon	+DSD	Backbone	+DSD	Backbone
Weather	96	**0.1744 ± 0.0007**	0.2045 ± 0.0010	**0.1792 ± 0.0016**	0.1816 ± 0.0005
	168	**0.2091 ± 0.0022**	0.2430 ± 0.0048	**0.2104 ± 0.0003**	0.2150 ± 0.0010
	336	**0.2777 ± 0.0007**	0.3017 ± 0.0022	0.2861 ± 0.0010	**0.2801 ± 0.0028**
	720	**0.3433 ± 0.0020**	0.3585 ± 0.0004	**0.3471 ± 0.0020**	0.3478 ± 0.0056
Electricity	96	**0.1805 ± 0.0001**	0.1953 ± 0.0001	**0.1811 ± 0.0001**	0.1947 ± 0.0005
	168	**0.1759 ± 0.0001**	0.1825 ± 0.0001	**0.1755 ± 0.0002**	0.1841 ± 0.0003
	336	**0.1887 ± 0.0002**	0.1971 ± 0.0000	**0.1891 ± 0.0004**	0.1967 ± 0.0005
	720	**0.2219 ± 0.0005**	0.2331 ± 0.0002	**0.2218 ± 0.0001**	0.2283 ± 0.0003

B Computational Complexity of the Frequency Decoupling Module

This appendix analyzes the computational cost of the Frequency Decoupling Module, covering spectrum conversion, sorting, Lorenz-based concentration analysis, CAU adjustment, mask generation, and reconstruction.

The input sequence is first transformed into the frequency domain using an rFFT. Applied to each channel of length L, the rFFT costs $O(CL \log L)$ and produces $F = L/2 + 1$ frequency bins.

Table 9: Performance comparison of backbone models with and without DSD.

Methods		SCINet		+DSD		DLinear		+DSD		PatchTST		+DSD		FITS		+DSD		iTransformer		+DSD		Fredformer		+DSD		FilterNet		+DSD		Amplifier		+DSD	
Metrics		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTh1	96	0.401	0.456	**0.361**	**0.425**	0.368	**0.423**	**0.361**	0.425	**0.360**	**0.423**	0.361	0.425	0.377	0.430	**0.361**	**0.426**	0.370	0.431	**0.363**	**0.426**	0.368	**0.426**	**0.367**	0.428	0.384	0.441	**0.364**	**0.427**	0.376	0.434	**0.365**	**0.427**
	168	0.475	0.509	**0.392**	**0.451**	0.398	**0.449**	**0.390**	0.450	0.395	**0.452**	**0.394**	**0.452**	0.413	0.459	**0.390**	**0.450**	0.399	0.456	**0.386**	**0.447**	0.403	0.455	**0.391**	**0.451**	0.419	0.469	**0.391**	**0.452**	0.413	0.464	**0.392**	**0.452**
	336	0.584	0.577	**0.444**	**0.490**	0.448	**0.485**	**0.441**	0.487	0.448	0.489	**0.440**	**0.484**	0.457	**0.483**	**0.441**	0.487	0.450	0.484	**0.434**	**0.481**	0.449	**0.480**	**0.442**	0.485	0.470	0.496	**0.445**	**0.488**	0.456	0.489	**0.445**	**0.487**
	720	0.682	0.635	**0.621**	**0.602**	**0.556**	**0.560**	0.607	0.594	0.582	0.578	**0.575**	**0.577**	**0.593**	**0.564**	0.608	0.594	0.600	**0.571**	**0.577**	0.578	0.615	0.580	**0.577**	**0.578**	0.623	0.581	**0.582**	**0.580**	0.613	**0.577**	**0.578**	0.578
ETTh2	96	0.135	0.274	**0.111**	**0.236**	0.111	0.237	**0.110**	**0.236**	0.118	0.246	**0.111**	**0.237**	0.117	0.240	**0.113**	**0.239**	0.128	0.255	**0.110**	**0.236**	0.113	0.237	**0.110**	**0.235**	0.122	0.248	**0.112**	**0.238**	0.114	0.240	**0.110**	**0.236**
	168	0.171	0.308	**0.128**	**0.254**	**0.126**	**0.253**	0.128	**0.253**	0.134	0.262	**0.126**	**0.251**	0.136	0.255	**0.128**	**0.253**	0.140	0.264	**0.126**	**0.252**	0.132	0.253	**0.127**	**0.251**	0.138	0.261	**0.126**	**0.250**	0.133	0.257	**0.125**	**0.250**
	336	0.183	0.322	**0.137**	**0.263**	0.139	0.270	**0.137**	**0.263**	0.142	0.269	**0.137**	**0.266**	0.148	0.269	**0.138**	**0.264**	0.158	0.282	**0.136**	**0.265**	0.141	**0.264**	**0.140**	0.267	0.154	0.277	**0.138**	**0.266**	0.150	0.273	**0.137**	**0.265**
	720	0.219	0.354	**0.163**	**0.290**	0.175	0.313	**0.163**	**0.288**	0.168	0.299	**0.162**	**0.290**	0.180	0.292	**0.162**	**0.287**	0.196	0.308	**0.161**	**0.290**	0.180	0.296	**0.160**	**0.289**	0.196	0.308	**0.163**	**0.295**	0.189	0.302	**0.162**	**0.294**
ETTm1	96	0.352	0.419	**0.339**	**0.397**	**0.312**	**0.382**	0.337	0.396	**0.326**	**0.394**	0.337	0.396	**0.320**	**0.389**	0.338	0.396	0.342	0.395	**0.335**	**0.394**	0.343	**0.395**	**0.334**	**0.395**	0.357	0.407	**0.338**	**0.396**	0.345	0.402	**0.339**	**0.397**
	168	0.397	0.448	**0.366**	**0.420**	**0.356**	**0.410**	0.368	0.420	**0.356**	**0.413**	0.363	0.418	**0.366**	**0.417**	0.368	0.420	0.375	0.422	**0.361**	**0.416**	0.366	0.418	**0.363**	**0.417**	0.382	0.427	**0.364**	**0.418**	0.376	0.424	**0.363**	**0.418**
	336	0.457	0.484	**0.422**	**0.453**	**0.417**	**0.447**	0.430	0.460	**0.414**	**0.450**	0.423	0.455	0.437	**0.458**	**0.430**	0.460	0.428	0.455	**0.418**	**0.452**	0.430	0.458	**0.422**	**0.454**	0.440	0.464	**0.422**	**0.455**	0.444	0.466	**0.423**	**0.455**
	720	0.562	0.553	**0.477**	**0.495**	**0.471**	**0.489**	0.475	0.492	**0.465**	**0.489**	0.471	0.490	0.499	0.500	**0.474**	**0.491**	0.500	0.505	**0.474**	**0.491**	0.499	0.506	**0.471**	**0.490**	0.507	0.509	**0.473**	**0.492**	0.510	0.512	**0.477**	**0.494**
ETTm2	96	**0.074**	**0.196**	0.075	**0.196**	0.080	0.203	**0.077**	**0.199**	0.081	0.204	**0.078**	**0.198**	0.081	0.203	**0.078**	**0.199**	**0.076**	**0.198**	0.077	0.199	0.080	0.201	**0.079**	**0.200**	0.082	0.204	**0.078**	**0.198**	0.080	0.201	**0.078**	**0.199**
	168	0.103	0.238	**0.092**	**0.217**	0.093	0.219	**0.092**	**0.218**	0.093	**0.218**	**0.092**	**0.218**	0.097	0.221	**0.092**	**0.219**	0.093	0.221	**0.092**	**0.219**	0.096	0.221	**0.093**	**0.218**	0.097	0.223	**0.092**	**0.218**	0.095	0.221	**0.093**	**0.218**
	336	**0.103**	**0.239**	0.115	0.242	**0.114**	0.245	0.116	**0.242**	**0.114**	**0.244**	0.116	0.245	0.120	0.244	**0.115**	**0.241**	0.122	0.253	**0.115**	**0.244**	0.119	0.246	**0.114**	**0.242**	0.121	0.247	**0.114**	**0.242**	0.120	0.247	**0.115**	**0.244**
	720	0.153	0.294	**0.140**	**0.266**	0.143	0.275	**0.140**	**0.266**	0.142	**0.267**	**0.139**	0.268	0.151	0.269	**0.140**	**0.266**	0.152	0.275	**0.139**	**0.268**	0.151	0.271	**0.139**	**0.268**	0.156	0.276	**0.139**	**0.268**	0.152	0.274	**0.140**	**0.265**
Weather	96	0.203	0.272	**0.175**	**0.220**	0.206	0.256	**0.174**	**0.217**	0.182	0.218	**0.177**	**0.215**	0.201	0.221	**0.174**	**0.217**	0.180	**0.204**	**0.176**	0.216	0.188	**0.209**	**0.177**	0.216	**0.175**	**0.203**	0.177	0.217	**0.170**	**0.198**	0.177	0.218
	168	0.246	0.303	**0.211**	**0.258**	0.245	0.301	**0.209**	**0.254**	0.215	0.254	**0.210**	**0.246**	0.235	**0.247**	**0.210**	0.256	0.218	**0.234**	**0.214**	0.255	0.226	**0.239**	**0.214**	0.256	**0.212**	**0.232**	0.215	0.257	**0.208**	**0.229**	0.211	0.248
	336	0.314	0.344	**0.279**	**0.311**	0.300	0.336	**0.277**	**0.308**	**0.279**	**0.302**	0.286	0.314	0.306	**0.295**	**0.278**	0.308	0.293	**0.288**	**0.284**	0.313	0.300	**0.291**	**0.285**	0.314	**0.286**	**0.283**	0.287	0.316	**0.283**	**0.281**	0.287	0.317
	720	0.367	0.370	**0.345**	**0.357**	0.359	0.382	**0.340**	**0.347**	**0.345**	0.358	0.347	**0.350**	0.364	**0.332**	**0.341**	0.348	0.358	**0.328**	**0.346**	0.350	0.364	**0.332**	**0.347**	0.350	0.351	**0.324**	**0.348**	0.352	0.350	**0.323**	**0.349**	0.353
Exchange	96	0.088	0.219	**0.055**	**0.170**	**0.052**	**0.164**	0.055	0.171	**0.054**	**0.168**	0.055	0.172	**0.053**	**0.165**	0.055	0.170	0.056	**0.169**	**0.055**	0.171	**0.053**	**0.164**	0.054	0.170	0.056	**0.172**	**0.055**	**0.172**	**0.053**	**0.165**	0.056	0.174
	168	0.128	0.263	**0.090**	**0.217**	0.089	0.217	**0.087**	**0.214**	0.185	0.341	**0.089**	**0.219**	0.087	0.215	**0.086**	**0.213**	**0.090**	**0.217**	**0.090**	0.219	**0.086**	**0.213**	0.090	0.220	0.092	0.223	**0.090**	**0.220**	**0.088**	**0.216**	0.090	0.220
	336	0.205	0.338	**0.174**	**0.306**	0.178	0.312	**0.164**	**0.297**	0.350	0.476	**0.166**	**0.301**	0.175	0.310	**0.164**	**0.297**	0.186	0.320	**0.166**	**0.302**	0.180	0.315	**0.166**	**0.302**	0.186	0.320	**0.166**	**0.302**	0.177	0.311	**0.167**	**0.303**
	720	0.419	0.490	**0.294**	**0.406**	0.362	0.461	**0.260**	**0.382**	0.604	0.633	**0.296**	**0.414**	0.406	0.498	**0.261**	**0.383**	0.387	0.484	**0.300**	**0.417**	0.385	0.481	**0.300**	**0.416**	0.481	0.554	**0.299**	**0.416**	0.406	0.501	**0.301**	**0.418**
Electricity	96	0.186	0.299	**0.180**	**0.265**	0.195	0.277	**0.180**	**0.265**	0.196	0.278	**0.181**	**0.265**	0.203	0.280	**0.180**	**0.265**	0.190	0.269	**0.181**	**0.265**	0.191	**0.271**	**0.185**	**0.271**	**0.181**	0.272	**0.181**	**0.265**	**0.177**	**0.265**	0.181	**0.265**
	168	0.204	0.312	**0.175**	**0.265**	0.183	0.272	**0.176**	**0.265**	0.184	0.271	**0.176**	**0.264**	0.187	0.272	**0.176**	**0.266**	0.184	**0.268**	**0.179**	0.269	0.184	0.270	**0.177**	**0.265**	0.181	0.273	**0.176**	**0.265**	**0.175**	**0.265**	0.176	0.266
	336	0.223	0.334	**0.188**	**0.283**	0.197	0.294	**0.189**	**0.283**	0.197	0.293	**0.189**	**0.284**	0.202	0.290	**0.189**	**0.284**	0.199	0.287	**0.190**	**0.285**	0.207	0.300	**0.190**	**0.286**	0.199	0.295	**0.191**	**0.286**	0.193	0.288	**0.191**	**0.286**
	720	0.252	0.355	**0.221**	**0.320**	0.233	0.332	**0.222**	**0.320**	0.228	0.326	**0.222**	**0.320**	0.239	0.324	**0.221**	**0.320**	0.240	0.325	**0.222**	**0.321**	0.251	0.339	**0.223**	**0.321**	0.244	0.337	**0.224**	**0.323**	0.237	0.330	**0.224**	**0.323**
Traffic	96	0.494	0.429	**0.401**	**0.331**	0.505	0.388	**0.400**	**0.331**	0.434	0.354	**0.410**	**0.338**	0.521	0.386	**0.400**	**0.331**	**0.397**	**0.332**	0.410	0.338	0.384	**0.318**	**0.382**	0.325	0.447	0.359	**0.412**	**0.340**	0.435	0.353	**0.416**	**0.342**
	168	0.446	0.382	**0.412**	**0.332**	0.479	0.366	**0.412**	**0.332**	0.450	0.356	**0.420**	**0.338**	0.495	0.366	**0.412**	**0.332**	0.427	0.342	**0.421**	**0.338**	**0.418**	0.340	0.421	**0.339**	0.452	0.357	**0.428**	**0.344**	0.443	0.351	**0.427**	**0.343**
	336	0.441	0.372	**0.437**	**0.346**	0.504	0.381	**0.437**	**0.346**	0.472	0.366	**0.446**	**0.352**	0.525	0.381	**0.437**	**0.346**	0.451	0.355	**0.446**	**0.352**	**0.445**	**0.352**	0.446	**0.352**	0.478	0.372	**0.451**	**0.356**	0.471	0.365	**0.450**	**0.355**
	720	0.495	0.399	**0.470**	**0.370**	0.533	0.408	**0.469**	**0.370**	0.495	0.386	**0.477**	**0.374**	0.552	0.402	**0.469**	**0.370**	0.482	0.380	**0.477**	**0.374**	0.488	0.388	**0.476**	**0.374**	0.514	0.397	**0.484**	**0.380**	0.505	0.390	**0.485**	**0.381**
1st Count		2	2	30	31	8	10	24	23	9	11	23	23	4	9	28	23	3	10	30	22	4	13	28	22	4	5	29	28	7	9	25	24
Avg		0.305	0.368	**0.265**	**0.327**	0.279	0.338	**0.263**	**0.325**	0.288	0.346	**0.265**	**0.326**	0.289	0.334	**0.263**	**0.325**	0.277	0.331	**0.264**	**0.326**	0.276	0.329	**0.264**	**0.326**	0.287	0.338	**0.266**	**0.328**	0.279	0.332	**0.267**	**0.328**

After removing the DC component, computing the magnitude over the $F \times C$ spectrum costs $O(CF)$. Sorting the F magnitudes for each channel then costs $O(CF \log F)$.

Based on the sorted spectrum, computing the cumulative magnitude, Lorenz curve, and concentration score costs $O(CF)$. The CAU also processes a length-F vector per channel, resulting in $O(CF)$ complexity.

The adaptive soft mask is computed from the cumulative top-k magnitude ratio and CAU output using element-wise operations and frequency-order restoration, resulting in $O(CF)$ complexity. Finally, irFFT transforms the masked and residual spectra back to the time domain with $O(CL \log L)$ complexity.

Combining all operations yields $O(CL \log L) + O(CF \log F) + O(CF)$. Since $F = O(L)$, the final time complexity of the Frequency Decoupling Module simplifies to $O(CL \log L)$.

C Implementation Details

Each CAU is implemented as a two-layer MLP with hidden size 32 and a final Tanh activation. Each auxiliary predictor $g^{(j)}$ is instantiated as a lightweight MLP and takes the decoupled component $\mathbf{x}^{(j)}_{\text{dec}}$ and the original input $\mathbf{x}$ as inputs. It first encodes $\mathbf{x}^{(j)}_{\text{dec}}$ through a small MLP with hidden dimension 64, concatenates the encoded features with $\mathbf{x}$, and then passes the fused representation through another MLP with hidden dimension 128 to produce the final prediction. We set d to 3 for ETT, 2 for Weather, 3 for ExchangeRate, 2 for Electricity, and 3 for Traffic.

D Dataset Summary

Table 6 summarizes the datasets used in our experiments. **ETT**[1] contains transformer load and oil-temperature measurements, with hourly subsets (**ETTh1**, **ETTh2**) and 15-minute subsets (**ETTm1**, **ETTm2**). **Weather**[2] contains 21 meteorological variables sampled every 10 minutes. **ExchangeRate**[3] contains daily exchange rates from eight countries. **Electricity**[4] contains hourly electricity consumption from 321 users. **Traffic**[5] contains hourly traffic measurements from 862 freeway sensors.

E Detailed Results for Frequency Decoupling Strategy

Table 7 reports the absolute MSE values corresponding to the relative improvements shown in Figure 5.

F Robustness Evaluation with Multi-seed Experiments

Table 8 reports five-seed results. DSD improves average MSE over the corresponding backbones, with small standard deviations across the evaluated datasets and horizons.

G Full Horizon-wise Forecasting Results

Table 9 reports the complete horizon-wise MSE and MAE results for all datasets and backbones. These results provide the detailed values underlying the dataset-wise averages summarized in Table 1.

[1] https://github.com/zhouhaoyi/ETDataset
[2] https://www.bgc-jena.mpg.de/wetter/
[3] https://github.com/laiguokun/multivariate-time-series-data
[4] https://archive.ics.uci.edu/ml/datasets/ElectricityLoadDiagrams20112014
[5] http://pems.dot.ca.gov

GenAI Usage Disclosure

Generative AI tools were used only for grammar checking and typo correction.

References

[1] Ali N. Akansu and Mark J. T. Smith (Eds.). 1996. *Subband and Wavelet Transforms: Design and Applications.* Springer New York, NY. doi:10.1007/978-1-4613-0483-8

[2] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. 2013. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432* (2013).

[3] George EP Box, Gwilym M Jenkins, Gregory C Reinsel, and Greta M Ljung. 2015. Time series analysis: forecasting and control. (2015).

[4] Peng Chen, Yingying ZHANG, Yunyao Cheng, Yang Shu, Yihang Wang, Qingsong Wen, Bin Yang, and Chenjuan Guo. 2024. Pathformer: Multi-scale Transformers with Adaptive Pathways for Time Series Forecasting. In *The Twelfth International Conference on Learning Representations.*

[5] David L Donoho. 1995. De-noising by soft-thresholding. *IEEE transactions on information theory* 41, 3 (1995), 613–627.

[6] Alireza Ermagun and David Levinson. 2018. Spatiotemporal traffic forecasting: review and proposed directions. *Transport Reviews* 38, 6 (2018), 786–814.

[7] Mojtaba A Farahani, MR McCormick, Robert Gianinny, Frank Hudacheck, Ramy Harik, Zhichao Liu, and Thorsten Wuest. 2023. Time-series pattern recognition in Smart Manufacturing Systems: A literature review and ontology. *Journal of Manufacturing Systems* 69 (2023), 208–241.

[8] Jingru Fei, Kun Yi, Wei Fan, Qi Zhang, and Zhendong Niu. 2025. Amplifier: Bringing attention to neglected low-energy components in time series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 39. 11645–11653.

[9] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* 9, 8 (1997), 1735–1780.

[10] Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, et al. 2023. Time-llm: Time series forecasting by reprogramming large language models. *arXiv preprint arXiv:2310.01728* (2023).

[11] Shruti Kaushik, Abhinav Choudhury, Pankaj Kumar Sheron, Nataraj Dasgupta, Sayee Natarajan, Larry A Pickett, and Varun Dutt. 2020. AI in healthcare: time-series forecasting using statistical, neural, and ensemble architectures. *Frontiers in big data* 3 (2020), 4.

[12] Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. 2021. Reversible instance normalization for accurate time-series forecasting against distribution shift. In *International conference on learning representations.*

[13] Ashish Kumar, Ramana Ranganatham, Rama Komaragiri, and Manjeet Kumar. 2019. Efficient QRS complex detection algorithm based on Fast Fourier Transform. *Biomedical engineering letters* 9, 1 (2019), 145–151.

[14] James Lee-Thorp, Joshua Ainslie, Ilya Eckstein, and Santiago Ontanon. 2021. Fnet: Mixing tokens with fourier transforms. *arXiv preprint arXiv:2105.03824* (2021).

[15] Shiyang Li, Xiaoyong Jin, Yao Xuan, Xiyou Zhou, Wenhu Chen, Yu-Xiang Wang, and Xifeng Yan. 2019. Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting. *Advances in neural information processing systems* 32 (2019).

[16] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. 2017. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. *arXiv preprint arXiv:1707.01926* (2017).

[17] Minhao Liu, Ailing Zeng, Muxi Chen, Zhijian Xu, Qiuxia Lai, Lingna Ma, and Qiang Xu. 2022. Scinet: Time series modeling and forecasting with sample convolution and interaction. *Advances in Neural Information Processing Systems* 35 (2022), 5816–5828.

[18] Xu Liu, Junfeng Hu, Yuan Li, Shizhe Diao, Yuxuan Liang, Bryan Hooi, and Roger Zimmermann. 2024. Unitime: A language-empowered unified model for cross-domain time series forecasting. In *Proceedings of the ACM Web Conference 2024.* 4095–4106.

[19] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. 2024. iTransformer: Inverted Transformers Are Effective for Time Series Forecasting. In *The Twelfth International Conference on Learning Representations.*

[20] Zhiding Liu, Mingyue Cheng, Zhi Li, Zhenya Huang, Qi Liu, Yanhu Xie, and Enhong Chen. 2023. Adaptive normalization for non-stationary time series forecasting: A temporal slice perspective. *Advances in Neural Information Processing Systems* 36 (2023), 14273–14292.

[21] Donghao Luo and Xue Wang. 2024. DeformableTST: Transformer for Time Series Forecasting without Over-reliance on Patching. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems.*

[22] Yuqi Nie, Nam H Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *The Eleventh International Conference on Learning Representations.*

[23] Xihao Piao, Zheng Chen, Taichi Murayama, Yasuko Matsubara, and Yasushi Sakurai. 2024. Fredformer: Frequency debiased transformer for time series forecasting. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining.* 2400–2410.

[24] David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. 2020. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International journal of forecasting* 36, 3 (2020), 1181–1191.

[25] Sonja Sauerbrei. 2009. Lorenz curves, size classification, and dimensions of bubble size distributions. *Entropy* 12, 1 (2009), 1–13.

[26] Donghwan Song, Adrian Matias Chung Baek, and Namhun Kim. 2021. Forecasting stock market indices using padding-based fourier transform denoising and time series deep learning models. *IEEE Access* 9 (2021), 83786–83796.

[27] U.S. Energy Information Administration. 2001. Residential Energy Consumption Survey (RECS), 2001. https://www.eia.gov/consumption/residential/data/2001/index.php?view=consumption. Accessed: 2025-09-25.

[28] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[29] Hao Wang, Lichen Pan, Yuan Shen, Zhichao Chen, Degui Yang, Yifei Yang, Sen Zhang, Xinggao Liu, Haoxuan Li, and Dacheng Tao. 2025. FreDF: Learning to Forecast in the Frequency Domain. In *The Thirteenth International Conference on Learning Representations.*

[30] Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. 2023. TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis. In *The Eleventh International Conference on Learning Representations.*

[31] Zhijian Xu, Ailing Zeng, and Qiang Xu. 2024. FITS: Modeling Time Series with $10k$ Parameters. In *The Twelfth International Conference on Learning Representations.*

[32] Weiwei Ye, Songgaojun Deng, Qiaosha Zou, and Ning Gui. 2024. Frequency Adaptive Normalization For Non-stationary Time Series Forecasting. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 31350–31379.

[33] Kun Yi, Jingru Fei, Qi Zhang, Hui He, Shufeng Hao, Defu Lian, and Wei Fan. 2024. Filternet: Harnessing frequency filters for time series forecasting. *Advances in Neural Information Processing Systems* 37 (2024), 55115–55140.

[34] Kun Yi, Qi Zhang, Wei Fan, Shoujin Wang, Pengyang Wang, Hui He, Ning An, Defu Lian, Longbing Cao, and Zhendong Niu. 2023. Frequency-domain mlps are more effective learners in time series forecasting. *Advances in Neural Information Processing Systems* 36 (2023), 76656–76679.

[35] Rose Yu, Stephan Zheng, Anima Anandkumar, and Yisong Yue. 2018. Long-term forecasting using tensor-train rnns. (2018).

[36] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. 2023. Are transformers effective for time series forecasting?. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 11121–11128.

[37] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 11106–11115.

[38] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. 2022. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *International conference on machine learning.* PMLR, 27268–27286.

[39] Tian Zhou, Ziqing Ma, xue wang, Qingsong Wen, Liang Sun, Tao Yao, Wotao Yin, and Rong Jin. 2022. FiLM: Frequency improved Legendre Memory Model for Long-term Time Series Forecasting. In *Advances in Neural Information Processing Systems*, Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho (Eds.).

[40] Tian Zhou, Peisong Niu, Liang Sun, Rong Jin, et al. 2023. One fits all: Power general time series analysis by pretrained lm. *Advances in neural information processing systems* 36 (2023), 43322–43355.